Séance 3

Rendre l'application intelligente et robuste

Lambdas · Streams · Exceptions · Journalisation 🔒

Objectifs de la séance

- **AA5** : transformer et agréger des données avec les lambdas et l'API Stream
- **AA6** : gérer les erreurs avec des exceptions métier, sans masquer les erreurs ni divulguer d'information
- **Plan** : 1. Lambdas · 2. Streams · 3. Exceptions et journalisation

> Le modèle de la séance 2 sait **stocker**. Aujourd'hui, il apprend à **répondre** et à **échouer proprement**.

1. Des interfaces aux lambdas

🟢 Point de départ : `Discount`

```
interface Discount {
    BigDecimal apply(BigDecimal total);   // UNE seule méthode abstraite
}

// Nouvelle promo "happy hour" : une classe entière pour une ligne utile
class HappyHourDiscount implements Discount {
    @Override
    public BigDecimal apply(BigDecimal total) {
        return total.subtract(new BigDecimal("2.00"))
                    .max(BigDecimal.ZERO);
    }
}
```

> `Discount` ne décrit pas un objet, mais un **comportement**.

📖 Support §1.1 et §1.2 — Discount, une classe pour une ligne

De la classe à la lambda

Classe anonyme (avant Java 8)

```
Discount happyHour = new Discount() {
    @Override
    public BigDecimal apply(
            BigDecimal t) {
        return t.subtract(TWO)
                .max(BigDecimal.ZERO);
    }
};
```

Lambda (Java 8 et +)

```
Discount happyHour =
    t -> t.subtract(TWO)
          .max(BigDecimal.ZERO);
```

- On ne garde que **paramètres** → **corps**
- Le compilateur déduit le type de `t` et la méthode `apply`

📖 Support §1.3 et §1.4 — Classe anonyme, lambda (TWO = 2.00 €)

Syntaxe d'une lambda

```
// Aucun paramètre
Supplier<String> newId = () -> "order-" + UUID.randomUUID();
// Un paramètre : parenthèses facultatives
Discount none = t -> t;
// Deux paramètres
Comparator<BigDecimal> byAmount = (a, b) -> a.compareTo(b);
// Plusieurs instructions : accolades et return
Function<BigDecimal, String> label = t -> {
    if (t.compareTo(new BigDecimal("50")) >= 0) return "grosse";
    return "standard";
};
```

> Une lambda implémente une **interface fonctionnelle** : une interface avec **une seule** méthode abstraite.

📖 Support §1.5 — Syntaxe et capture de variables

🤔 Est-ce que ça compile ?

```
int pendingCount = 0;
orders.forEach(o -> {
    if (o.getStatus() == OrderStatus.PENDING) {
        pendingCount++;
    }
});
IO.println(pendingCount);
```

Réponse : non, une lambda ne modifie pas une variable locale

`error: local variables referenced from a lambda expression must be final or effectively final`

- Une lambda peut **lire** une variable locale *effectivement finale*, pas la **modifier**
- Elle peut s'exécuter plus tard, ou sur un autre thread (séance 4)

```
// ✅ Pas de bricolage (int[], AtomicInteger) : on laisse l'API compter
long pendingCount = orders.stream()
        .filter(o -> o.getStatus() == OrderStatus.PENDING)
        .count();                                              // 1
```

📖 Support §1.5 — Capture de variables

Les interfaces fonctionnelles du JDK

Interface	Méthode	Rôle	Exemple FoodFast
`Predicate<T>`	`boolean test(T)`	tester	commande en attente ?
`Function<T, R>`	`R apply(T)`	transformer	commande → total
`Consumer<T>`	`void accept(T)`	agir	notifier le client
`Supplier<T>`	`T get()`	fournir	nouvel identifiant
`Comparator<T>`	`int compare(T, T)`	comparer	trier par total

> Inutile d'inventer vos interfaces : `java.util.function` en fournit une quarantaine.

Les interfaces fonctionnelles sur FoodFast

```
Predicate<Order> isPending = o -> o.getStatus() == OrderStatus.PENDING;
Function<Order, BigDecimal> toTotal = o -> o.getTotal();
Consumer<Order> notify =
        o -> IO.println("Notification envoyée pour " + o.getStatus());
Comparator<Order> byTotal =
        (a, b) -> a.getTotal().compareTo(b.getTotal());

List<Order> orders = new ArrayList<>(platform.getOrders());
orders.sort(byTotal);                  // List.sort attend un Comparator
Order cheapest = orders.getFirst();
IO.println(toTotal.apply(cheapest));  // 9.80
IO.println(isPending.test(cheapest)); // true
```

📖 Support §1.6 — Exemple complet et composition (and, negate, thenComparing)

🟢 Références de méthodes : `Integer::sum`, enfin expliqué

```
// Séance 2, Order.addDish : merge(clé, valeur, fusion(ancienne, nouvelle))
items.merge(dish, quantity, (oldQty, qty) -> oldQty + qty);
items.merge(dish, quantity, (a, b) -> Integer.sum(a, b));
items.merge(dish, quantity, Integer::sum);          // les 3 sont égales
```

Référence	Lambda équivalente
`Order::getTotal`	`o -> o.getTotal()`
`BigDecimal::add`	`(a, b) -> a.add(b)`
`platform::getOrderById`	`id -> platform.getOrderById(id)`

📖 Support §1.7 — Les quatre formes de références de méthodes

2. L'API Stream

Boucle ou stream ?

Impératif : *comment* faire

```
List<BigDecimal> totals =
        new ArrayList<>();
for (Order o : orders) {
    if (o.getStatus()
            != CANCELLED) {
        totals.add(o.getTotal());
    }
}
totals.sort(
        Comparator.reverseOrder());
```

Stream : *quoi* obtenir

```
List<BigDecimal> totals =
    orders.stream()
        .filter(o -> o.getStatus()
                != CANCELLED)
        .map(Order::getTotal)
        .sorted(Comparator
                .reverseOrder())
        .toList();
```

> Le stream se lit comme la question : « les totaux des commandes non annulées, du plus grand au plus petit ».

Anatomie d'un pipeline

```
orders.stream()          source
    .filter(...)         intermédiaire  ┐  rendent un Stream,
    .map(...)            intermédiaire  ┘  ne calculent rien encore
    .toList()            terminale      →  déclenche le calcul
```

- Un stream **ne stocke rien** : c'est un tuyau, pas un récipient
- Il **ne modifie jamais** sa source (`filter` ne retire rien de la `Map`)
- **Une seule** opération terminale, à la fin

`filter`, `map`, `toList`

```java
// Dans DeliveryPlatform
public List<Order> findOrdersByStatus(OrderStatus status) {
    return orders.values().stream()
                 .filter(o -> o.getStatus() == status)
                 .toList();
}

// Les clients dont une commande est en route
List<String> names = platform.getOrders().stream()
        .filter(o -> o.getStatus() == OrderStatus.IN_DELIVERY)
        .map(o -> o.getCustomer().getName())
        .toList();                                    // [Bob]
```

📖 Support §2.3 — filter, map, toList

`sorted` et `limit` : le top 3 des commandes

```
List<Order> top3 = orders.stream()
        .filter(o -> o.getStatus() != OrderStatus.CANCELLED)
        .sorted(Comparator.comparing(Order::getTotal).reversed())
        .limit(3)
        .toList();
// Bob : 37.50 · Alice : 30.50 · Bob : 23.80
```

- `Comparator.comparing(clé)` fabrique le comparateur, `.reversed()` l'inverse
- `BigDecimal` est trié avec `compareTo` : correct pour un montant (S2)

📖 Support §2.4 — sorted, limit, Comparator

🟢 Répondre à une question : `count`, `anyMatch`, `findFirst`

```
long pending = orders.stream()
        .filter(o -> o.getStatus() == OrderStatus.PENDING)
        .count();                                          // 1

BigDecimal limit = new BigDecimal("35");
boolean anyBig = orders.stream()
        .anyMatch(o -> o.getTotal().compareTo(limit) > 0); // true

Optional<Order> preparing = orders.stream()
        .filter(o -> o.getStatus() == OrderStatus.PREPARING)
        .findFirst();                                      // vide
```

📖 Support §2.5 — count, anyMatch, allMatch, noneMatch, findFirst

🟢 Chiffre d'affaires : `reduce` avec `BigDecimal`

```
BigDecimal revenue = orders.stream()
        .filter(o -> o.getStatus() == OrderStatus.DELIVERED)
        .map(Order::getTotal)
        .reduce(BigDecimal.ZERO, BigDecimal::add);       // 71.70
```

```
ZERO.add(30.50)  = 30.50
30.50.add(17.40) = 47.90
47.90.add(23.80) = 71.70
```

- Pas de `sum()` pour `BigDecimal`, et **jamais** de `double` pour l'argent
- `BigDecimal.ZERO` : point de départ, et résultat si le flux est vide

📖 Support §2.6 — reduce avec BigDecimal

Commandes par statut : `groupingBy`

```
Map<OrderStatus, Long> countByStatus = orders.stream()
        .collect(Collectors.groupingBy(
                Order::getStatus,           // la clé de chaque groupe
                Collectors.counting()));    // le calcul dans chaque groupe

countByStatus.get(OrderStatus.DELIVERED);  // 3
countByStatus.get(OrderStatus.PREPARING);  // null : pas de groupe vide
```

- Autres calculs par groupe : `toList()` (par défaut), `summingInt`, `reducing`

> L'ordre d'une `Map` issue de `groupingBy` n'est **pas garanti** : ne testez jamais son affichage.

📖 Support §2.7 — groupingBy, counting, EnumMap

🔵 Plats les plus commandés : `flatMap`

```
Map<String, Integer> qtyByDish = orders.stream()
        .filter(o -> o.getStatus() != OrderStatus.CANCELLED)
        .flatMap(o -> o.getItems().entrySet().stream()) // à plat
        .collect(Collectors.groupingBy(
                e -> e.getKey().name(),
                Collectors.summingInt(Map.Entry::getValue)));

qtyByDish.entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue()
                         .reversed())
        .limit(3)
        .forEach(e -> IO.println(e.getKey() + " x" + e.getValue()));
// Margherita x5 · Burger x3 · Tiramisu x2
```

📖 Support §2.9 — flatMap ; §2.10 — toMap et les collisions de clés

🤔 Prédisez la sortie

```
List<String> statuses = List.of("PENDING", "DELIVERED", "CANCELLED");

Stream<String> s = statuses.stream()
        .filter(st -> {
            IO.println("filtre " + st);
            return st.startsWith("D");
        });
IO.println("pipeline construit");

IO.println(s.toList());
```

Réponse : un stream est paresseux

```
pipeline construit
filtre PENDING
filtre DELIVERED
filtre CANCELLED
[DELIVERED]
```

- Les opérations intermédiaires **décrivent**, la terminale **exécute**
- Sans opération terminale, rien ne se passe, sans erreur ni avertissement
- Un stream ne sert **qu'une fois** : un second `s.toList()` lève une `IllegalStateException`

📖 Support §2.8 — Les trois pièges des streams

🟢 Pièges : effets de bord et liste non modifiable

❌ Remplir une liste externe

```
List<Order> pending =
        new ArrayList<>();
orders.stream()
    .filter(o -> o.getStatus()
            == PENDING)
    .forEach(pending::add);
```

✅ Laisser le stream construire

```
List<Order> pending =
    orders.stream()
        .filter(o -> o.getStatus()
                == PENDING)
        .toList();
```

- `toList()` rend une liste **non modifiable** : `add` lève `UnsupportedOperationException`
- Besoin de la modifier ? `new ArrayList<>(stream.toList())`

📖 Support §2.8 et §2.3 ; §2.11 — streams parallèles

3. Exceptions et journalisation

La hiérarchie des exceptions

```
Throwable
├── Error                  grave (JVM) : on ne l'attrape pas
└── Exception              CHECKED : le compilateur oblige à la traiter
    ├── IOException, SQLException…         (monde extérieur)
    └── RuntimeException   UNCHECKED : aucune obligation
        ├── IllegalArgumentException       (mauvais paramètre)
        │   └── NumberFormatException
        ├── IllegalStateException          (mauvais moment)
        └── OrderNotFoundException         (la nôtre)
```

- **Checked** : `throws IOException` dans la signature, ou `try` / `catch`
- **Unchecked** : bug ou contrat violé, on corrige l'appelant

📖 Support §3.1 et §3.2 — Pourquoi des exceptions, checked ou unchecked

🟢 `try` / `catch` / `finally`

```
try {
    var request = new DeliveryRequest(Integer.parseInt(qtyText),
                                      LocalDateTime.parse(dateText));
    return "OK : " + request;
} catch (NumberFormatException | DateTimeParseException e) {
    return "Saisie mal formée, vérifiez la quantité et la date.";
} catch (IllegalArgumentException e) {     // APRÈS sa sous-classe
    return "Saisie refusée : la quantité doit être positive.";
} finally {
    IO.println("(saisie traitée)");         // toujours exécuté
}
```

> On attrape là où l'on **sait quoi faire** de l'erreur, du **plus spécifique** au plus général.

📖 Support §3.3 — try/catch/finally, multi-catch, pièges

try-with-resources

```
static List<Dish> loadMenu(Path file) throws IOException {
    try (Stream<String> lines = Files.lines(file)) {
        return lines.filter(l -> !l.isBlank())
                    .map(l -> parseLine(l))
                    .toList();
    }   // lines.close() est appelé automatiquement, même en cas d'erreur
}
```

- Pour tout `AutoCloseable` : fichier, connexion réseau, connexion JDBC (S4)
- Plusieurs ressources : fermées dans l'ordre **inverse** d'ouverture
- `IOException` est checked : `loadMenu` la déclare, l'appelant décidera

📖 Support §3.4 — try-with-resources

🤔 Prédisez la sortie

```
class Ticket implements AutoCloseable {
    Ticket() { IO.println("ouverture"); }
    public void close() { IO.println("fermeture"); }
}

try (Ticket t = new Ticket()) {
    IO.println("préparation");
    throw new IllegalStateException("four en panne");
} catch (IllegalStateException e) {
    IO.println("catch : " + e.getMessage());
} finally {
    IO.println("finally");
}
```

Réponse : la ressource est fermée avant le `catch`

```
ouverture
préparation
fermeture
catch : four en panne
finally
```

> Quand on traite l'erreur, la ressource est **déjà libérée**.

📖 Support §3.4 — Ordre de fermeture, exceptions supprimées 🔵

🟢 Exception métier : `OrderNotFoundException`

```
class OrderNotFoundException extends RuntimeException {
    private final String orderId;
    OrderNotFoundException(String orderId) {
        super("Commande introuvable : " + orderId);
        this.orderId = orderId;
    }
    public String getOrderId() { return orderId; }
}
```

Pourquoi unchecked ?

- L'appelant ne peut pas réparer sur place : l'erreur remonte à la frontière
- Il pouvait l'éviter avec `findOrderById` : c'est un **contrat violé**
- Une checked ne passerait pas dans `map(platform::getOrderById)`

📖 Support §3.5 — Exception métier, checked ou unchecked

`getOrderById` ou `findOrderById` ?

```java
/** Recherche : l'absence est un résultat normal. */
public Optional<Order> findOrderById(String id) {
    return Optional.ofNullable(orders.get(id));
}

/** Accès : la commande DOIT exister. */
public Order getOrderById(String id) {
    return findOrderById(id)
            .orElseThrow(() -> new OrderNotFoundException(id));
}
```

- Barre de recherche du client → `findOrderById` : « aucun résultat »
- Le livreur valide **sa** livraison → `getOrderById` : sinon, anomalie

📖 Support §3.6 — Lever ou renvoyer un Optional

Transition interdite : `IllegalStateException`

```
public void moveTo(OrderStatus next) {
    if (!status.canMoveTo(next)) {          // règle portée par l'enum
        throw new IllegalStateException(
                "Transition interdite : " + status + " -> " + next);
    }
    // ...
    status = next;                          // modifié APRÈS vérification
}

order.moveTo(OrderStatus.CANCELLED);
order.moveTo(OrderStatus.IN_DELIVERY);
// IllegalStateException: Transition interdite : CANCELLED -> IN_DELIVERY
```

- `IllegalArgumentException` : **le paramètre** est invalide
- `IllegalStateException` : **l'objet** n'est pas dans le bon état

📖 Support §3.7 — Valider les transitions

🟢 Ne jamais avaler une exception

```
// ❌ Le catch vide
catch (IllegalStateException e) {
}

// ❌ La valeur magique
catch (NumberFormatException e) {
    return 0;          // ou null
}

// ❌ La console, et on continue
catch (IOException e) {
    e.printStackTrace();
}
```

```
// ✅ 1. Ne pas attraper
// ✅ 2. Valider en amont
// ✅ 3. Traiter vraiment
//       (refuser, annuler)
// ✅ 4. Envelopper, avec la cause
catch (NumberFormatException e) {
    throw new IllegalArgumentException(
            "Pourcentage invalide", e);
}
```

> Une erreur masquée devient une **donnée fausse**, découverte trop tard, loin de sa cause.

📖 Support §3.8 — Ne jamais avaler ; §3.9 — Envelopper sans perdre la cause

Journaliser : `System.Logger`, pas `printStackTrace`

```
private static final Logger LOG =
        System.getLogger(DeliveryService.class.getName());

void markDelivered(String orderId) {
    Order order = platform.getOrderById(orderId);
    try {
        order.moveTo(OrderStatus.DELIVERED);
        LOG.log(Level.INFO, "Commande {0} livrée", order.getId());
    } catch (IllegalStateException e) {
        LOG.log(Level.WARNING, "Livraison refusée : " + order.getId(), e);
        throw e;                    // on journalise ET on relance
    }
}
```

📖 Support §4.1 — System.Logger, niveaux, pièges

🔒 Fuite d'information par les messages d'erreur (CWE-209)

❌ Le détail technique part chez le client

```
} catch (RuntimeException e) {
    return "500 " + e;
}
```

```
500 java.lang.IllegalState…
Connexion refusée : jdbc:
postgresql://db-prod-02:5432
/foodfast (user=ff_admin)
```

✅ Détail dans les logs, référence au client

```
} catch (RuntimeException e) {
    String ref = UUID.randomUUID()
                     .toString();
    LOG.log(Level.ERROR,
            "Incident " + ref, e);
    return "500 Une erreur est "
         + "survenue. Réf. " + ref;
}
```

📖 Support §4.2 — CWE-209, réponses d'erreur

🔒 Ce qu'on ne met pas dans les logs

```
// ❌ Données personnelles : les logs sont copiés, archivés, partagés
LOG.log(Level.INFO, "Livraison pour " + c.getEmail()
        + ", " + c.getAddress());

// ✅ Un identifiant suffit pour enquêter
LOG.log(Level.INFO, "Livraison de {0} pour le client {1}",
        order.getId(), c.getId());
```

- Jamais : mots de passe, jetons, numéros de carte, données de santé
- Attention au `toString()` d'un `record` : il affiche **tous** ses champs
- RGPD : principe de minimisation des données

📖 Support §4.3 — Données personnelles dans les logs

🔒 Injection dans les logs (CWE-117)

```
// L'identifiant vient de l'URL : l'attaquant y glisse un \n
String orderId = "order-42\nINFO: Remboursement de 500 EUR valide";
LOG.log(Level.WARNING, "Commande introuvable : " + orderId);  // ❌
```

```
WARNING: Commande introuvable : order-42
INFO: Remboursement de 500 EUR valide          <- fausse ligne !
```

```
static String forLog(String untrusted) {                    // ✅
    return untrusted.replaceAll("\\p{Cntrl}", "_");
}
```

📖 Support §4.4 — Injection dans les logs, Log4Shell

🟢 Fichiers : `Path` désigne, `Files` agit

```
Path menu = Path.of("data").resolve("menu.csv");      // data/menu.csv, rien sur le disque
Files.createDirectories(menu.getParent());
Files.writeString(menu, "Margherita;12.50\n");          // UTF-8, remplace le fichier
List<String> lines = Files.exists(menu) ? Files.readAllLines(menu) : List.of();
```

```
try (OutputStream out = new BufferedOutputStream(Files.newOutputStream(menu))) {
    out.write("Burger;11.90\n".getBytes(StandardCharsets.UTF_8));
}   // sans close() : le tampon n'est jamais écrit, fichier vide
```

📖 Support §5.1 à §5.3 — Path, Files, flux d'octets

🔵 Sauvegarder sans jamais tout perdre

```
Path tmp = Files.createTempFile(file.getParent(), "save-", ".tmp");  // même dossier
try {
    Files.writeString(tmp, content);
    Files.move(tmp, file, ATOMIC_MOVE, REPLACE_EXISTING);   // d'un seul coup
} finally {
    Files.deleteIfExists(tmp);
}
```

- Tué au milieu d'un `writeString` : fichier vide, ancienne sauvegarde perdue
- Avec le renommage atomique : l'ancienne version **ou** la nouvelle, jamais un mélange
- 🔒 Nom de fichier venu de l'extérieur : `normalize()` puis `startsWith(dossier)` (CWE-22)
- Tests : `@TempDir`, un dossier neuf pour chaque test

📖 Support §5.4 à §5.6 — écriture atomique, CWE-22, @TempDir

Ce qu'il faut retenir

- **Lambda** : implémentation concise d'une interface à une méthode ; `Integer::sum`, `Order::getTotal`
- **Stream** : source → intermédiaires paresseuses → **une** terminale ; `toList()` non modifiable
- Montants : `reduce(BigDecimal.ZERO, BigDecimal::add)` ; groupes : `groupingBy` + `counting`
- `OrderNotFoundException` (unchecked) si la commande **doit** exister, sinon `Optional`
- **Jamais** de `catch` vide, de `return 0` ou `null` par défaut, de `printStackTrace`
- 🔒 Détail dans les logs, message neutre à l'utilisateur ; ni données personnelles ni `\n` bruts
- Fichiers : `Path` + `Files`, flux fermés par try-with-resources, sauvegarde par renommage atomique

Pour aller plus loin

- 🔵 Support §2.9 à §2.11 : `flatMap`, `toMap` et collisions de clés, streams parallèles
- 🟣 Support §2.12 : Gatherers (JDK 24), lots de livraisons avec `windowFixed`
- 🟣 Support §3.10 et §3.11 : exceptions checked dans les lambdas, variable `_`
- Annexe A du support : le modèle FoodFast complet de la séance
- dev.java · OWASP Logging et Error Handling Cheat Sheets · SEI CERT (ERR00-J, ERR01-J)

> Séance 4 : concurrence et JDBC, sous l'angle « le bug et la faille ».